

Background: Smart Parking Barrier System

A local university wants to implement a smart parking barrier system at its main gate. The system should:

- Automatically open the barrier for registered staff and students.
- Count the number of cars in the parking lot.
- Display the number of available spots.
- Trigger an alert if someone tries to enter when the parking is full.

The goal is to simulate and implement the logic using the Integrated Problem-Solving Process.

Step 1: Understand and Define the Problem (Analyse)

The university needs a smart parking barrier system that allows registered staff and students to enter automatically, counts the vehicles inside the parking lot, displays the number of available spaces, and alerts security when an unauthorized vehicle tries to enter or when the parking lot is full.

Step 2: Organise and Describe the Data

Automate access, vehicle count, alert for full capacity.

Inputs: RFID/ID of user, sensor for car entry/exit, max capacity

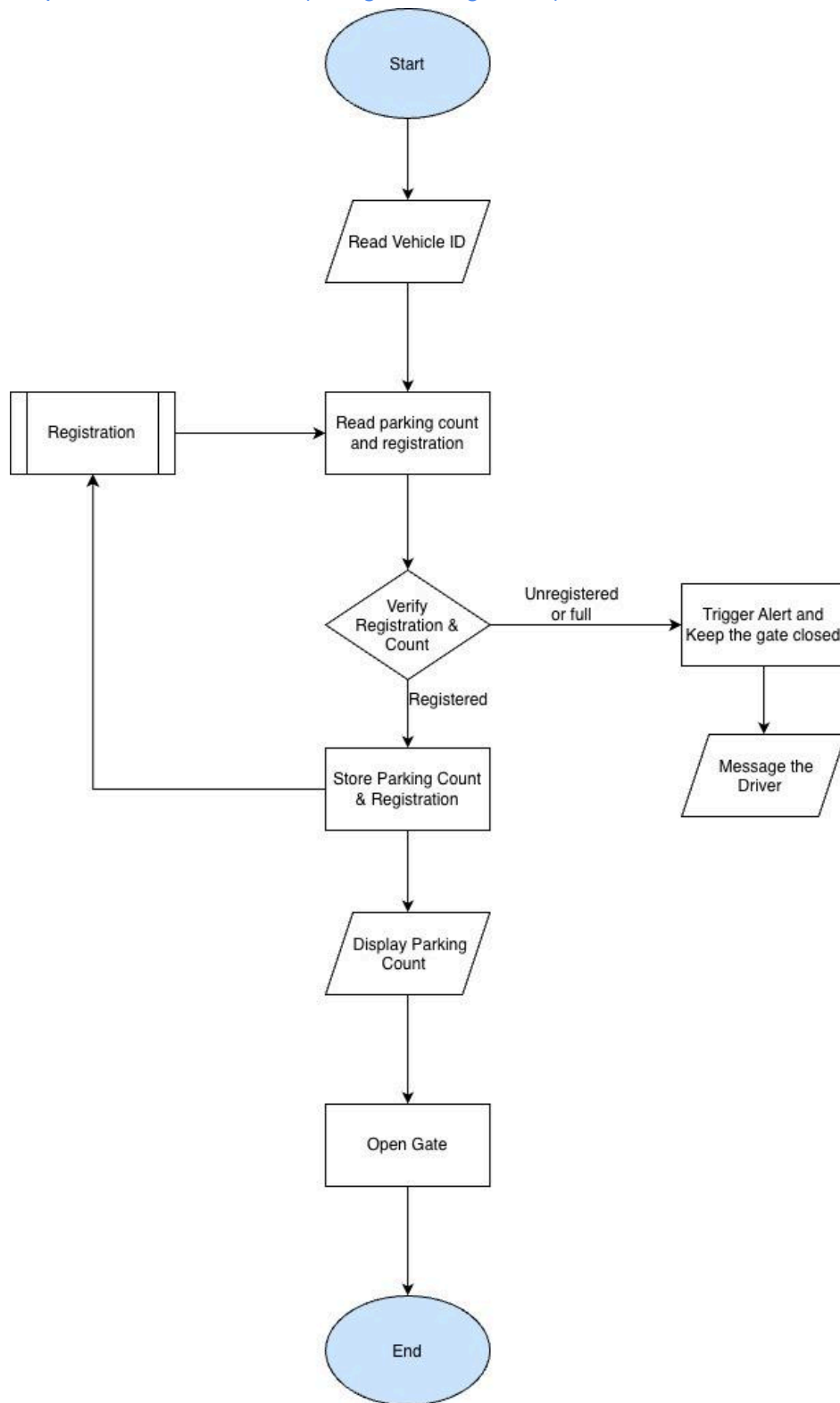
output: barrier control(open/close)

Assumptions: each vehicle has unique ID, sensors are reliable, capacity fixed.

Expected output: Automatic barrier control, updated vehicle count, and a Full Capacity alert when the car park is full.

Parameter	Type	Example Values
Max Capacity	Constant	100
Current count	variable	75 vehicle
Vehicle ID	Input	A132D
Is registered	Decision	Yes/No
Barrier control	Output	Open/Close
Display available	Output	25 Available
Alert trigger	Output	On/Off

Step 3: Plan the Solution (Design the Algorithm)



Step 4: Implement the Solution (Word Coding)

- Once upon a time
- Read vehicle ID(input)
- Check if the vehicle is registered
- If not registered, slam the boom gate over the bumper of the car(That is don't open the gate)
- If registered but carpark is full - Do not Open
- If registered and there is enough space then open the gate, yey!!!
- Note that all the transaction are stored in a database called "registrations".

Step 5: Test and Refine the Solution (Debug and Verify)

Manual test case

Assumption

Max no of cars allowed: 100

Registrations database is up to date (Actual car park spots availability)

Test for

Registered user with 95 cars- Allow entry

Registered user with 101 cars - alerts + entry locked + message

Integrate a web dashboard for monitoring

Add override for emergency vehicles

Part 5:AI Agent Integration

1.Improved Word Code

1. Read Vehicle ID
2. Check registration status
3. If NOT registered → Alert ON, Gate CLOSED
4. If registered → Check parking count
5. If full → Alert ON, Gate CLOSED
6. If space available → Increment count, Open gate, Display available spots
7. Store transaction
8. End

2.Propose different ways to solve the problem or enhance your flowchart logic.

- Event-Driven Logic — sensors trigger actions
- State Machine Design — Idle → Checking → Full → OpenGate
- Queue-Based Access Control — vehicles processed one at a time
- License Plate Recognition — camera-based verification
- Cloud-based Monitoring — dashboard + remote alerts

3.Hardware Components

- RFID reader (RC522 or PN532)
- IR/Ultrasonic sensors for entry/exit

- Servo motor for barrier
- LCD/LED display
- Raspberry Pi for database + dashboard
- Arduino for real-time control

Architecture

- Arduino handles sensors + gate movement
- Raspberry Pi handles database, logging, web dashboard
- Communication via Serial, WiFi, or MQTT

4. Improve documentation

Help write a professional README.md or project summary.

A strong README.md includes:

- Project overview
- Features
- System architecture
- Flowchart
- Word code
- Hardware list
- Setup instructions
- Future improvements